This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

Lauca: a Workload Duplicator for Benchmarking Transactional Database Performance

Siyang Weng, QingShuai Wang, Luyi Qu, Rong Zhang, Peng Cai, Weining Qian, Aoying Zhou

***Abstract*—Generating synthetic workloads is essential and critical to the performance evaluation of database systems. When benchmarking database performance for a specific application, the similarity between synthetic workloads and real application workloads determines the credibility of the evaluation results. However, it meets a great challenge to catch workload characteristics for a target online transaction processing (OLTP) application considering the complexity of transaction executions. To address this problem, we propose a workload duplicator (Lauca) that can generate synthetic workloads with highly similar performance metrics compared to a specific application on both centralized and distributed databases. By carefully studying the application-driven workload generation problem, we present *Transaction Logic*, *Data Access Distribution* and *Partition Access Distribution* to characterize runtime workloads and propose novel generation algorithms to guarantee the high fidelity of synthetic workloads. To the best of our knowledge, Lauca is the first application-driven transactional workload generator. We conduct extensive experiments based on TPC-C, SmallBank and YCSB on both centralized and distributed databases. The experimental results show that Lauca consistently generates high-quality synthetic workloads.**

***Index Terms*—Performance Evaluation, Synthetic Workload, OLTP Applications**

I. Introduction

Performance evaluation is essential to the development of database management systems (DBMSs), and evaluation results are important for analyzing problems or making choices. Application-driven synthetic workload generation aims to construct similar workloads to real application workloads, which are used to evaluate database capability for a *specific* application [1], [2]. It has wide application scenarios, including:

Database selection. Application developers need to select an appropriate DBMS with satisfying performance for their target workloads. It is an effective way to evaluate candidate DBMSs with test workloads similar to real application workloads.

PoC testing. PoC (proof of concept) testing is crucial for database vendors when marketing their products to applications. Creating a test scenario closely resembling the application provides the strongest PoC validation.

Application-driven performance optimization. Database performance may not be as good as the expectation for online workloads. But for database vendors, developers cannot reproduce the performance bugs to launch the optimization for lack of a real application scenario.

Siyang Weng, QingShuai Wang, Luyi Qu, Rong Zhang (corresponding author), Peng Cai, Weining Qian and Aoying Zhou are with School of Data Science and Engineering, East China Normal University, China. E-mail:{syweng, qswang, luyiqu}@stu.ecnu.edu.cn, {rzhang, pcai, wnqian, ayzhou}@dase.ecnu.edu.cn.

Currently, we usually use real application workloads, standard benchmark workloads, or synthetic workloads to benchmark database performance. But existing approaches are sub-optimal for application-driven database performance evaluation [2]–[4].For database developers, replicating the application scenario from their clients for performance evaluation is usually impossible for the data protection issue, and then it challenges us to confirm database performance [5]. Standard benchmarks [6]–[8] cannot accurately represent the characteristics of any application or all applications, and thus the results are not sufficient to assess a database's suitability for any given application. Though OLTP-Bench [9] is implemented to evaluate various benchmarks for DBMSs, it is limited by the scenarios depicted in benchmarks. These scenarios may not be the same as the in-production scenario, leading to evaluation deviation. So, generating application-driven synthetic workloads for performance evaluation is vital for databases. And it has attracted significant attention in online analytical processing (OLAP) applications [2], [3], [10]. However, it is still an unexplored area of OLTP applications.

In this paper, we propose a transactional workload duplicator **Lauca** for application-driven database performance evaluation. *Lauca* aims to generate a synthetic scenario, i.e., database and workloads, for any real application and the scenarios defined in benchmarks. The essential goal is that synthetic workloads generated by *Lauca* are as similar as possible to the application workloads in terms of runtime performance metrics, e.g., *throughput*, *latency*, or *resource usage*. To achieve this goal, we propose *Transaction Template* to specify the sketch of actions in target applications for both the stored procedure [11] and the interactive mode. We define *Transaction Logic* and *Access Distribution* to characterize the key workload characteristics, i.e., execution semantics, which are determined by the critical runtime transaction performance on databases. *Transaction Logic* is proposed for the first time to catch the latent business logic based on the key transaction structure and the parameter dependencies inside transactions. *Access Distribution* includes *Data Access Distribution* and *Partition Access Distribution*. Specifically, *Data Access Distribution* portrays various and changing data access of workload; *Partition Access Distribution* describes the characteristics of partition access on distributed databases. *Access distribution* strongly influences database performance for the expensive I/O cost of disk access or network cost of distributed communication, if any.

Lauca isolates the production environment from the generation environment for data protection. Components involving real application data/workload traces, which are used to extract

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

workload or data characteristics, are only executed in the production environment by data owners.

To the best of our knowledge, *Lauca* is the first transactional workload generator for application-driven database performance evaluation. We introduce *Transaction Logic*, *Data Access Distribution*, and *Partition Access Distribution* to represent the runtime workload characteristics of OLTP applications. We have made great efforts in designing generation algorithms for synthesizing both workloads and data and guaranteeing the similarity of execution semantics between generated workloads and application workloads. Extensive experiments are launched. The results show that *Lauca* can effectively characterize the runtime execution semantics, and the deviations in overall performance metrics between the synthetic workloads generated by *Lauca* and the real application workloads are consistently less than 6%.

This paper is a significant extension of our conference version [12]. We extend the approach to 1) provide a general way to simulate *data access continuity*(§ IV-C) with *Continuous Data Access Distribution*, 2) design a two-level hierarchical data structure for catching *Partition Access Distribution* and accomplish the partition access control (§V), 3) provide the theoretical analysis and formalize the relationship between the backward length with cache size, recorded data size, database size and the acceptable simulation deviation (Appendix A and §IV-C), and 4) launch more experiments to verify the effectiveness of our new design with in-depth analysis of the results (§VII-C and §VII-D).

II. Preliminaries

A. Problem Definition

We first enumerate the requirements of application-driven workload generation as follows. And Table I shows some common notations.

TABLE I
NOTATIONS

Notations	Description
TX	transaction template
O_i	the ith operation
$p_{i,j}/r_{i,j}$	the jth parameter/return item in O_i
V	# of parameters or return items in a transaction template
H	schema
$D(\widetilde{D})$	real (generated) data characteristics
$W(\widetilde{W})$	real (generated) workload characteristics
P	performance metrics

Fidelity. Generated workloads should be highly similar to real application workloads on a specific database, measured by the *deviations* in performance metrics. *Performance metrics* contains *throughput*, *latency*, *physical resources utilizations* and so on. The higher the *similarity*, the smaller the *deviation*.

Scalability. The target application may have a large data scale and high *concurrency* or *throughput*. It requires the workload generator to be scalable to multiple nodes and can support database and workload generation in parallel.

Extensibility. When the *concurrency* or *throughput* of current real application workloads cannot meet test requirements, the generator should be able to extend the data and workloads of the real application to the desired scale.

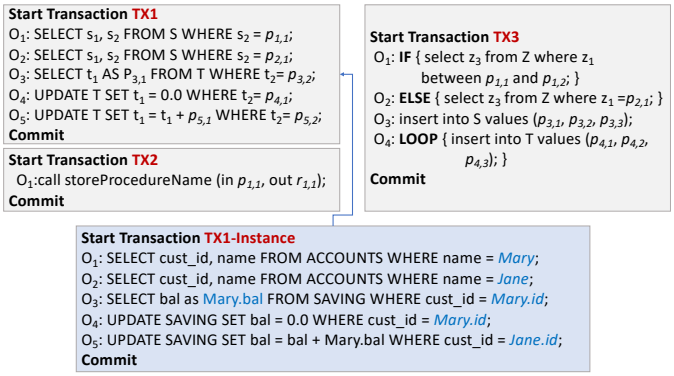


Fig. 1. Examples of transaction templates TX_1-TX_3 and a transaction instance TX_1-$Instance$ with respect to TX_1

Definition 1. ***Transaction Template:*** *It is a transaction sketch that consists of SQL operations (e.g., insert) and branch/loop structures (if any), represented by TX. But the table semantics are concealed by symbolizing parameter values and anonymizing table/column names in SQLs.*

Transactions in real applications are abstracted as a set of **Transaction Template**s defined in Definition 1. Both two mainstream transaction execution modes, i.e., an interactive mode and the stored procedure, can be represented by transaction templates. We present three sample transaction templates in Figure 1. TX_1 and TX_3 are executed interactively, while TX_2 is executed as a stored procedure. TX_1 contains five SQL operations, i.e., O_1 to O_5, and TX_3 contains branch and loop structures. For TX_2, its transaction template contains only one SQL operation, i.e., a stored procedure call. The ith operation in TX is O_i and $p_{i,j}$ (resp., $r_{i,j}$) is the jth parameter (resp., returned item) of O_i. For example, $p_{5,2}$ in TX_1 is the 2nd parameter in O_5. The number of all parameters and return items is V. And table/column names are anonymized, i.e., S, T, Z, s_i, t_j and z_k. All relationships among SQL operations are concealed, including parameters and returned items, as well as the judgment conditions of branches and the loop numbers of loop structures. Then we formulate the problem of application-driven workload generation for transactional databases as follows.

Definition 2. ***Application-driven synthetic workload generation:*** *Given a database DB and a specific application with schema H, data characteristics D, workload characteristics W, and performance metrics P, OLTP workload generation aims to generate a set of synthetic workloads $\widetilde{W}$ and data $\widetilde{D}$. Synthetic workloads should meet the requirement of fidelity, i.e., 1) $\widetilde{D}$ strictly follows D, 2) running $\widetilde{W}$ on DB filled with generated $\widetilde{D}$ yields the performance metrics as P; and 3) the generation algorithms can generate large-scale data/workloads for various DBMSs to satisfy scalability and extensibility.*

Specifically, in Definition 2, H defines the structure of tables, including the number of columns, the data type of each column, key/foreign key constraints as well as their referencing relationships with others, and the partition columns with their partition rules. D defines the domains of columns, the cardinality of values, and the average/maximum lengths for

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

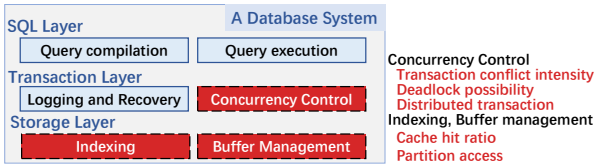


Fig. 2. Workload characteristics in red are critical for simulation if given a database

varchar typed columns. W includes a set of transaction templates and their runtime characteristics, including *Data Access Distribution* and *Partition Access Distribution*. P represents the performance of the applications, e.g., TPS.

B. Application Sensitive Workload Characteristics

A DBMS mainly consists of three components as in Figure 2, i.e., ***SQL Layer***, ***Transaction Layer*** and ***Storage Layer***. Though workload characteristics are closely related to the design of a database engine, we focus on workload simulation for a **specific application** to have the same performance on the **same DBMS** used in production. So, some factors that strongly influence performance are settled, such as database schema in H, transaction templates in W, the implementation mechanisms of a DBMS, etc. The emphasis lies in simulating the transaction characteristics specific to an application. Additionally, we explore the contrasting characteristics of centralized and distributed DBMSs, highlighting their distinctions.

Since transactional workloads usually involve a few rows of data and query optimizations affect the performance slightly, the generation strategies for synthetic workloads have little impact on *SQL Layer*, unlike query for analytical processing [13].

On *Transaction Layer*, Concurrency Control dictates transaction scheduling and significantly influences performance, particularly for high contention workloads. *Transaction conflict intensity* and *deadlock/rollback possibility* are the main concerned factors in both centralized and distributed DBMSs [14]. For a distributed DBMS, the additional scheduling of distributed transactions has always had a negative impact on performance for more network or disk I/O cost. So the proportion of distributed transactions (*distributed transaction ratio*) and the number of partitions accessed in a transaction (*partition access cardinality*) should be simulated to guarantee the target performance [15]–[18].

Log volume is the main determinant for the performance of *Logging and Recovery* [19]. It is determined by transaction templates in W, data characteristics in D, and the database engine. Since these factors are settled, log volumes are settled.

Indexing and *Buffer Management* are two main modules on *Storage Layer* [20]. With settled database schema in H and transaction templates in W, the index access cost is highly related to *access distribution* and *data distribution*. And buffer management cost is sensitive to *access distribution* and *data access continuity*. These factors affect *contention* and *cache hit ratio*, related to *data access skewness* and continuity. The physical index locality impacts the *cache hit ratio* as well, particularly in cases of index changes, such as structure modification in B-trees. However, in distributed DBMSs, data are partitioned and distributed among nodes according to the sharding rules. Accessing distributed partitions may lead to high network I/O. To guarantee a similar performance in a distributed environment, besides *partition access cardinality* on *Transaction Layer*, $Lauca$ expects to simulate workload with the same *partition access skewness* on *Storage Layer*.

C. Overview of Lauca

$Lauca$ generates synthetic data $\widetilde{D}$ together with the synthetic workloads $\widetilde{W}$ to simulate an application. The overall architecture is shown in Figure 3, which isolates **Production Environment** (**PE**) from **Evaluation Environment** (**EE**).

In **PE**, $Lauca$ takes workload traces (e.g., logs) and database statistics queried from the database as inputs, through a read-only connection. Then, $Lauca$ provides four components to extract characteristics from these inputs. Based on workload traces, *Transaction Logic Analyzer* abstracts transaction templates and logics inside transactions, e.g., parameter dependencies; *Access Distribution Analyzer* analyzes the access distribution of transactions containing pivotal SQL parameters to locate data and partition. These two components catch the execution semantics of workload in W. *Performance Monitor* fills *throughput info* in P. *Data Characteristics Extractor* gets the statistics for the schema H and data characteristics D, while H contains the structure of tables and D contains basic data statistics. All the detailed characteristics for schema H, data D and workload W have been explained below Definition 2 in § II-A. Each component only gets the characteristics mentioned above with its access authority restricted to data owners for data protection concerns. Then they are fed into **EE** through *Isolation Layer*. *Isolation Layer* further conceals sensitive information by anonymization, noise data insertion, and domain fuzzification, if needed. W, H, D, and P are then used to generate data $\widetilde{D}$ and workloads $\widetilde{W}$ by *Database Generator* and *Workload Generator* respectively. The extraction in **PE** and the generation in **EE** are isolated and automatically completed.

1) Workload Generation: Transaction conflict intensity, *deadlock possibility*, *distributed access* and *cache hit ratio* are application sensitive characteristics as analyzed in Figure 2. To keep the consistency among the synthetic workloads and the real application workloads in these four database-side workload characteristics, we define and manipulate three application-side workload characteristics, namely ***Transaction Logic*** (§III), ***Data Access Distribution*** (§IV) and ***Partition Access Distribution*** (§V) extracted by *Transaction Logic Analyzer*, and *Access Distribution Analyzer*. The inputs of *Transaction Logic Analyzer* are workload traces of a whole workload cycle. Workload traces can be logged by application clients or the database server. *Transaction Logic* depicts the dependency relationship among SQL parameters and returned items. It represents the latent business logic inside target applications and greatly influences *conflict intensity* and *deadlock possibility*. *Data Access Distribution* characterizes the dynamics of SQL operations access distribution on data, namely the data distributions of SQL parameters. It has a great influence on *cache hit ratio* as well as *transaction conflict intensity* and *deadlock possibility*. *Partition Access Distribution* depicts the parameter and the transaction distributions on logical

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

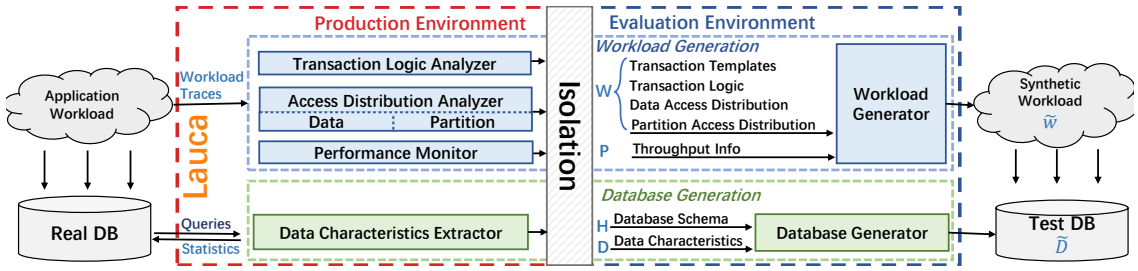


Fig. 3. The overall architecture and workflow of Lauca

partitions. It affects the cost of network communication and distributed transaction scheduling for distributed databases. Though in some database implementations, *Transaction Layer* and *Storage Layer* are merged, factors affecting performance are still consistent with our previous analysis.

For workload generation, transaction templates are part of the inputs. By instantiating all the symbol parameters with concrete values in a transaction template, we can get an instantiated transaction for the test database. To guarantee the similarity between the synthetic and the real application workload, parameter instantiation in the workload generator should satisfy the requirements of *Transaction Logic*, *Data Access Distribution*, and *Partition Access Distribution*.

2) Database Generation: Database generation involves creating tables with integrity constraints and some data characteristics of columns, e.g., data types, table sizes, and distinct value cardinality. Then, $Lauca$ defines random functions for each column to satisfy basic data characteristics as Touchstone [21], e.g., string length. Though the parameters instantiated for transactions are closely related to the generated data, in OLTP applications, the performance is highly related to the semantics of transactions instead of the result size, which is usually small and does not dominate performance. So, $Lauca$ provides a coarse way to simulate the size of returned results by considering the column cardinality and data distribution.

The column generator contains an index generator and a value transformer. For the key column, the index generator simply generates integer typed indexes for primary keys sequentially and selects foreign keys from them. It ensures the uniqueness of primary keys and the referential integrity of foreign keys. For the non-key column, the index generator generates indexes according to the desired data characteristics, e.g., its distinct value cardinality. Different transformers are constructed to deterministically map indexes to synthetic values of any data type. For example, for a string type key column, there are N_S ($N_S > 0$) seed strings pre-generated, which satisfy the length requirements. We first select a seed string based on the index i, such as the $(i\%N_S)^{th}$ seed. Then we concatenate the index and the selected seed string as a unique output value. So applying the value transformer is the only difference between generating integer-typed values and the values of other types. Note that the value transformer for key columns should be injective, which guarantees the uniqueness of primary keys.

III. Transaction Logic

We propose ***Transaction Logic*** to represent the latent business logic for OLTP applications instead of the predicate logic [22]. We first give a concrete example to expose the importance of *Transaction Logic*. Then we give a formal definition. Finally, we present the method to extract *Transaction Logic* from workload traces.

A. Example for Transaction Logic

We use transaction instance TX_1-$Instance$ corresponding to transaction template TX_1 in Figure 1 to illustrate the necessity of *Transaction Logic*, which is an example transaction of bank transferring in SmallBank. In TX_1-$Instance$, Mary moves all her funds to Jane. In the workload, parameter dependencies reflect the business logic, e.g., $p_{4,1} = p_{3,2}$, $p_{5,1} = p_{3,1}$ and $p_{5,2} \neq p_{4,1}$. Suppose Table *ACCOUNTS* and *SAVING* in TX_1-$Instance$ (resp. S and T in TX_1) have been deployed in a distributed database and both of them are hash-partitioned by primary keys, i.e., $cust_id$ (resp. s_1 and t_2 in TX_1). Since $p_{4,1}$ and $p_{5,2}$ represent two different accounts in this business, we will have a low probability (e.g., 1%) to meet $p_{4,1} = p_{5,2}$. So O_4 and O_5 in TX_1 may cause distributed transactions in a high probability. Besides, parameters $p_{3,2}$ and $p_{4,1}$ of TX_1 are always of the same account and avoid any deadlocks between O_3 and O_4. This example illustrates that the latent business logic of the target application is concealed within parameters, i.e., the *Transaction Logic* we introduced. It has a substantial impact on database performance and must be consistently replicated in the synthesized workload. However, the common random parameter instantiation method cannot guarantee these parameter dependencies [14], [15].

B. Definition of Transaction Logic

Transaction Logic is defined by *transaction structure information* and *parameter dependency information*. Transaction structure information covers branch and loop structures with an important influence on transaction execution costs. Each branch is extracted with its execution probability and each loop is associated with its average loop number.

The relationship among SQL parameters and returned items in a transaction describes latent runtime execution semantics among/in SQL operations. Even if the relationship among parameters of a stored procedure call may affect its execution cost. To simplify the problem, we focus on the dependency between the current parameter $p_{i,j}$ and the previous parameters

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

$p_{m,n}$(or returned items $r_{u,v}$) ($u < i$ or $m \leqslant i$), which is met in a probability pr. We investigate the existing OLTP benchmarks and actual application workloads, and define four types of parameter dependencies influencing transaction execution costs. Firstly, **Equal Relationship** (**ER**) is the most common one, represented as $[p_{m,n}/r_{u,v}, p_{i,j}, ER, pr]$. It declares $p_{i,j}$ is equal to $p_{m,n}$ (or $r_{u,v}$). Secondly, **Inclusive Relationship** (**IR**) is represented as $[r_{u,v}, p_{i,j}, IR, pr]$. It reveals the value of $p_{i,j}$ is an element in the returned results of the previous SQL, i.e., $r_{u,v}$. Thirdly, **Linear Relationship** (**LR**) is represented as $[p_{m,n}/r_{u,v}, p_{i,j}, LR, (a, b)]$. It refers to that $p_{m,n}$ (or $r_{u,v}$) and $p_{i,j}$ can be represented by a linear function with a and b as the coefficients. If O_i is in a loop structure, $p_{m,n}$ linearly evolves from $p_{i,j}$ during the successive runs of the loop. Fourthly, **Between Relationship** (**BR**) is represented as $[p_{i,l}, p_{i,j}, BR, \Delta]$ with $j < l$. It is defined for predicates like '$col\ between\ p_{i,j}\ and\ p_{i,l}$', in which the value of col in O_i is between $p_{i,l}$ and $p_{i,j}$ and $p_{i,l}$=$p_{i,j} + \Delta$ with Δ as the increment size. All these four types affect *cache hit ratio*. Furthermore, **ER** widely influences the quantities of *distributed transactions* and *contention*; **LR** exists widely in transactions to launch simple operations to the returned values of a previous statement.

Notice that dependencies are not an exhaustive set and can be extended if needed. Based on the analysis, we formally define *Transaction Logic* in Definition 3.

Definition 3. ***Transaction Logic:*** *Given a transaction template with parameters and returned results in/from SQL operations O,* Transaction Logic *consists of transaction structures among operations in O and dependencies among parameters and returned results together with the existence probabilities for both structures and dependencies.*

Let us take the example transaction templates in Figure 1 to demonstrate the dependencies defined in Definition 3. From the previously mentioned bank transferring transaction instance, i.e., TX_1-$Instance$, when instantiating its transaction template TX_1, we should guarantee three ER typed dependencies, i.e., [$p_{4,1}$, $p_{3,2}$, ER, 100%], [$p_{5,1}$, $p_{3,1}$, ER, 100%] and [$p_{5,2}$, $p_{4,1}$, ER, 1%]. TX_3 contains a branch structure and a loop structure. A BR dependency locates in TX_3 between $p_{1,1}$ and $p_{1,2}$. During the loop execution, if $p_{4,1}$ remains unchanged and $p_{4,2}$ is increased by 1, we then get two ***LR*** typed dependencies, i.e., [$p_{4,1}$, LR, 100%, $(1, 0)$] and [$p_{4,2}$, LR, 100%, $(1, 1)$] for $p_{4,1}$ and $p_{4,2}$, respectively.

C. Dependency Extraction Algorithm

Transaction Logic represents application business logic and generally remains stable over time. So, it is possible to sample a portion of workload traces instead of all for extracting *Transaction Logic*. Since the *Transaction Logic* analysis is independent for each transaction template, the following extraction algorithm consists of six steps executed in parallel. The input is a transaction template with the corresponding workload traces of K sampled transaction instances. The average number of operations in transaction templates is N_o.

Step 1: Structure information. By counting the operation executions in the transaction template, probabilities are calculated for each branch and the average loop execution numbers, if any. The complexity is O(KN_o).

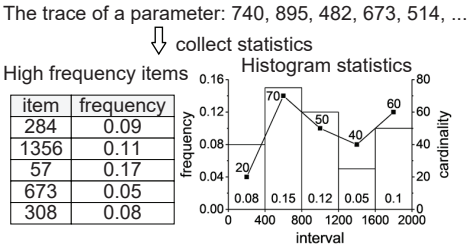


Fig. 4. An example for illustrating S-Dist

Step 2: Identify BR. We identify all parameter pairs, that satisfy the *BR* relationship in transaction templates, e.g., <$p_{i,l}$, $p_{i,j}$>, and traverse workload traces to get the average increment Δ for each pair. Then we construct *BR* typed dependencies for $p_{i,j}$. The complexity is also O(KN_o).

Step 3: Collect statistics for ER and IR. For every parameter $p_{i,j}$ in the transaction template, we traverse its previous parameters $p_{m,n}$ and returned items $r_{u,v}$. We count the occurrences where pairs, e.g., <$p_{m,n}$, $p_{i,j}$> (resp., <$r_{u,v}$, $p_{i,j}$>), satisfy the *ER* (resp., *ER* or *IR*) dependency. Thus, there are less than $V^2/2$ pairs. The complexity is O(KV^2).

Step 4: Collect statistics for LR. *LR* only involves numeric typed parameters and returned items. Since calculating coefficients (a, b) of an *LR* dependency requires two transaction instances, we randomly pair N_l groups of transaction instances from K instances. This maintains *LR* dependency relationships among parameters while simplifying computations. The complexity is O(N_lV^2) with $N_l \ll K^2$.

Step 5: Dependency filtering. With the statistics collected in Steps 3-4, we can easily construct *ER*, *IR*, and *LR* dependencies for each parameter $p_{i,j}$. In order to remove noises and alleviate calculation complexity, we filter out dependencies with low probabilities by checking each pair of dependencies. The complexity is O(N_o+V^2+N_lV^2) with N_o, V^2 and N_lV^2 as the maximum dependencies from Steps 2-4.

Step 6: Construct dependency for loop structure. The parameter dependencies may evolve gradually inside a loop in a linear way, i.e., *LR* dependency. To characterize parameter evolutions in a loop structure, the coefficient calculation is similar to that in Step 4 with the same complexity, but this kind of dependency is maintained independently for the loop structure. Notice that if the dependency construction in Steps 3-4 encounters an operation in the loop structure, only the trace for the first round of execution in the loop is used.

For each parameter $p_{i,j}$ in a transaction template TX, its probability pr for a dependency relationship with a previous parameter $p_{m,n}$ or returned item $r_{u,v}$ in TX ($m \leqslant i$ or $u < i$) is calculated as $pr = \frac{t_1}{t_2}$, with t_1 and t_2 as the size of transaction instances having the relationship and the total size of transaction instances for TX, respectively.

In summary, the complexity of our extraction algorithm is dominated by Steps 2-4, which is O(KN_o+KV^2+N_lV^2). In practice, *Transaction Logic* is stable and can be caught by a small set of traces. So K and N_l can be set to tens of thousands, and V and N_o may be tens or hundreds. Finally, the extraction of *Transaction Logic* is very fast.

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

IV. Data Access Distribution

We catch three kinds of *Data Access Distribution* on workloads, which are skewness, dynamics, and continuity. They influence database performance [9]. *Data Access Distribution* of workloads is determined by the values of parameters in transaction templates. Synthetic workload generation is actually an instantiation of those parameters. We use the workload traces containing pivotal parameters to index the data involved in SQL operations for extracting *Data Access Distribution*.

A. Skewed Data Access Distribution

Without loss of generality, we assume the form of predicates for OLTP workloads as '$col\ op\ p$', that is when doing op action, the parameter p determines *Data Access Distribution* on column col. We use *High Frequency Items* (**HFI**) and *Histogram Statistics* (**HS**) extracted from workload traces to represent **S**kewness of data access **D**istribution (*S-Dist*) for each parameter. In *S-Dist*, *HFI* records N_H hottest data items (i.e., concrete parameter values) with the corresponding access frequencies. The column domain is evenly divided into I intervals. In each interval, the distinct parameter values, except those in *HFI*, are used to construct the access frequency of each value and the access cardinality, which form *HS*. Even though the frequency of an interval is low, its small access cardinality can still cause high conflicts. So, both frequency and access cardinality are used to accurately catch access skewness for intervals. Figure 4 shows an example of *S-Dist* for an integer typed column with domain (0, 2000] and data cardinality 400. Both N_H and I are set as 5 here. Since new data generators are defined for database generation based on data characteristics (as in §II-C2), items collected in *HFI* and *HS* are not consistent with our generated database. Therefore, data transformation is needed to guarantee that instantiated transactions can access the database successfully. Since the index generator in database generation fills the data domain to meet the cardinality requirement, each index maps a distinct value and must exist. We propose to design an index generator with the same data cardinality domain and use the same value transformer as the database generation. Specifically:

1) For *HFI*, we simply construct an index generator based on the data cardinality. For this example column, we define an index generator as 'index=ranInt[1, 400]', which selects an index located in the cardinality domain of our generated database perfectly. Then we use the same value transformer as in database generation, i.e. '$value$=index$\times$5', to distribute the index (in [1,400]) uniformly to the column domain ((0,2000]). For example, for the hottest item 57 in *HFI*, we first get a random index, e.g., 39, from the index generator and then transform the index 39 to 195 by the value transformer. So we make 195 (in the generated database in Figure 5) to be a hot item with the same access frequency 0.17 as 57 (in the original database in Figure 4).

2) For *HS*, to simulate the access cardinality in each interval, we redefine the index generator but use the same value transformer. Suppose for the ith interval, it accesses cdn_i distinct values and its minimum/start index is minIdx_i. The average access cardinality in each interval is cdn_{avg}. We can create an

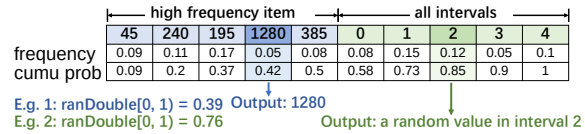


Fig. 5. An example of parameter generation using data generator

index generator 'index = $\lfloor \frac{\text{ranInt}[0,\ \text{cdn}_i)}{\text{cdn}_i} \times \text{cdn}_{avg} \rfloor + \text{minIdx}_i$' to generate cdn_i distinct values for the ith interval. For example, in Figure 5, the index generator for interval 2 is 'index = $\lfloor \frac{\text{ranInt}[0,\ 50)}{50} \times 80 \rfloor + 161$', with $\text{cdn}_2 = 50$, $\text{cdn}_{avg} = \frac{400}{5} = 80$, $\text{minIdx}_2 = 80 \times 2 + 1 = 161$.

To ensure generated values (indexes) conform to the desired frequency distribution, we calculate a cumulative probability array based on the frequencies of items in *HFI* and intervals in *HS* ('cumu prob' in Figure 5). Then, random values are generated between 0 and 1 and mapped to the cumulative probability array by a binary search to get the appropriate transformed values for parameter instantiation. In Figure 5, if the random number is 0.39 (E.g. 1), 1280 is output; if it is 0.76 (E.g. 2) located in interval 2, one of the values in this interval is output. The complexity of parameter instantiation is $O(\log(N_H + I)$ dominated by the binary search.

Although the parameter value in the example is of an integer type, our approach is generic. By converting other types to numeric types, the representation of *S-Dist* and the parameter instantiation method are exactly the same. For example, we can convert a string to an integer by its hash code h, and its interval is $h\%I$. For parameters on key columns, there is little difference. Since the primary key in our synthetic database is sequential, the domains of key columns in the synthetic database may differ from that in the real application database. Therefore, when collecting statistics for *S-Dist*, we use the domains of key columns in the real application database to divide the size of intervals and construct *HS*. But during workload generation, we use the domains of key columns in the synthetic database to support parameter instantiation.

B. Dynamic Data Access Distribution

S-Dist cannot accurately depict the dynamic distribution of data access. Supposing a workload updates a table in a time period and all updates access only one record in each second by a stable throughput, *S-Dist* is then uniform and has no hot data. However, in each second, one of the records is hot and leads to intensive conflicts and low performance. So, we propose *D-Dist* to characterize both **D**ynamics and skewness of data access **D**istribution.

We divide the workload traces of a parameter into multiple equal-length time windows according to log timestamps. The recommended time window size is one second for catching second-level workload changes. For parameter traces in each time window, we generate its individual *S-Dist*. *D-Dist* is then defined by a list of time-ordered *S-Dist*s. During generating synthetic workloads, we instantiate symbolic parameters following the *S-Dist* for each time window.

C. Continuous Data Access Distribution

Data may be accessed continually for a time period. Then the hot data is distributed with time dependency. This is

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

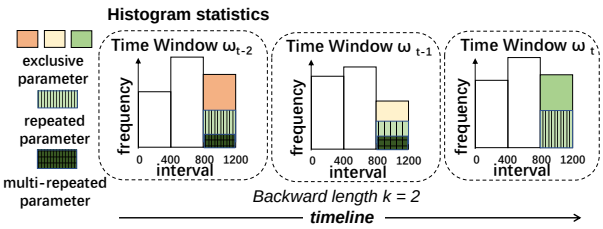


Fig. 6. An example for data statistics in continuous time windows

called **C**ontinuity of data access **D**istribution (*C-Dist*). For example, for online food delivery applications, steamed buns are frequently ordered in the morning, and coffee may be the favorite in the afternoon. Previously, *D-Dist* is defined to catch the skewness of data access in each time window independently, while ignoring the continuity between successive time windows. When *D-Dist* is used to generate synthetic workloads, the data accessed between successive time windows may be completely different, resulting in a lower *cache hit ratio*. However, *cache hit ratio* greatly impacts database performance, and cache tunning is usually launched to improve performance based on the data access pattern of workloads. So the simulation of *cache hit ratio* is imperative for performance simulation. *C-Dist* is defined to represent the repetition ratio of accessed data between successive time windows based on *D-Dist*.

To collect the statistics for *C-Dist*, the repetition ratio of accessed data is counted based on the previous time windows for each time window. The number of adjacent time windows considered backward is named as *backward length*, noted as k. For example, in Figure 6, we plot *D-Dist* for each time window by 3 histograms. If $k = 2$, the repetition ratios are collected between the current time window ω_t and its two previous time windows, i.e., ω_{t-2} and ω_{t-1} for the same histograms. The proportion of repetitive values is marked by the green vertical strip for the 3rd histogram among the three time windows. It is expected to ensure the repetition ratios for all histograms in each time window to guarantee *cache hit ratio* during workload generation. Since *HFI* is also included in *D-Dist*, *HFI* is treated in the same way as the histograms in *HS*, i.e., counting the repetitions in *HFI* and instantiating the parameter values in simulation accordingly.

Specifically, it takes two steps to instantiate parameters $w.r.t$ *cache hit ratio*. First, Algo. 1 is designed to collect the repetition statistics for each histogram in the time window by looking back k nearest previous windows. Second, the repetitive parameter values are generated according to the repetition ratio. The repetitive existence of data is considered instead of its frequency according to the cache management. But a parameter value may exist in multiple windows. For example, the 3rd interval in window ω_t has repetitive data with either window ω_{t-1} or ω_{t-2}, and some data exist in all three windows. Algo. 1 reduces this duplication during statistics.

Algo. 1 is designed to record repetitive parameter values in each histogram of the current window ω_t. In Algo. 1, for a $\omega_t \in \mathcal{W}$, we calculate its repetition ratio for its mth histogram h_m based on k previous nearest neighbor windows, i.e., $\omega_{t-1} \cdots \omega_{t-k}$. First, $\mathcal{R}_m$ is initialized to record repetitive parameter values in h_m (*line 3*). Then, we extract all distinct

Algorithm 1 Repetition Ratio Calculation

Input: Parameters in traces Φ, backward length k and windows $\mathcal{W}$.
Output: Repetition ratios Ψ; $\Psi_{i,j,m}$ is the differential repetition ratio for window w_i compared to w_j in histogram h_m.

1: **for all** time window $\omega_t \in \mathcal{W}$ **do**
2: **for all** histogram $h_m \in \omega_t$ **do**
3: Initialize a set $\mathcal{R}_m \leftarrow \varnothing$.
4: $\Phi_{t,m} \in \Phi \leftarrow$ all parameters in h_m of ω_t.
5: **for** i = 1 to k **do**
6: $\Phi_{t-i,m} \in \Phi \leftarrow$ all parameters in h_m of ω_{t-i}.
7: $\Delta_m \leftarrow \Phi_{t,m} \cap \Phi_{t-i,m} \backslash \mathcal{R}_m$.
8: $\Psi_{t,t-i,m} \leftarrow \frac{|\Delta_m|}{|\Phi_{t,m}|}$.
9: $\mathcal{R}_m \leftarrow \mathcal{R}_m \cup \Delta_m$.
10: **return** Ψ

parameter values in h_m of window ω_t as $\Phi_{t,m}$ (*line 4*). After that, the repetition ratio is calculated backward to k previous time windows (*line 5-10*). For each histogram h_m in a previous time window ω_{t-i}, its involved parameter values are extracted to $\Phi_{t-i,m}$ (*line 6*). The new repetitive values found (not be in $\Phi_{t,m}$) are kept to Δ_m (*line 7*). The repetition ratio is calculated as the ratio of the size of repetitive parameter values in Δ_m to the size of values in $\Phi_{t,m}$ (*line 8*). After obtaining the repetition ratio, $\mathcal{R}_m$ is updated to introduce the new repetitive parameter values in Δ_m (*line 9*). If the average cardinality for parameters in each time window is $\mathcal{C}$ and the backward length is k, the complexity of Algo. 1 is $O(k \cdot \mathcal{C})$.

Algorithm 2 Restoring Repetition Ratio

Input: Windows $\mathcal{W}$, backward length k, parameters in traces Φ, and differential repetition ratio Ψ with $\Psi_{i,j,m}$ as the repetition ratio from window ω_i to ω_j in histogram h_m.
Output: Parameters Ω, while $\Omega_{i,m}$ represents the parameters in histogram h_m of window ω_i.

1: **for all** time window $\omega_t \in \mathcal{W}$ **do**
2: **for all** histogram $h_m \in \omega_t$ **do**
3: Initialize a list $\Omega_{t,m} \leftarrow \varnothing$.
4: **for** $i = k$ to 1 **do**
5: $\Phi_{t,m} \in \Phi \leftarrow$ All parameters in I_m of ω_t.
6: $\mathcal{C}_{i,m} \leftarrow |\Phi_{t,m}| \times \Psi_{t,t-i,m}$.
7: $\mathcal{R} \leftarrow$ select $\mathcal{C}_{i,m}$ values from $\Omega_{t-i,m} \backslash \Omega_{t,m}$
8: $\Omega_{t,m} \leftarrow \Omega_{t,m} + \mathcal{R}$
9: Initialize a list $\mathcal{R} \leftarrow \varnothing$
10: **while** $|\Omega_{t,m}| + |\mathcal{R}| < |\Phi_{t,m}|$ **do**
11: $p \leftarrow$ Parameters generated by column generator.
12: **if** $p \notin \Omega_{t,m}$ **then**
13: $\mathcal{R} \leftarrow \mathcal{R} + p$
14: $\Omega_{t,m} \leftarrow \Omega_{t,m} + \mathcal{R}$
15: **return** Ω

Restoring Repetition Ratio for each histogram h_m of window ω_t is presented in Algo. 2, when instantiating parameters in transaction templates. Firstly, an empty space $\Omega_{t,m}$ is initialized to store the candidate parameters (*line 3*) for a histogram h_m in window ω_t. To instantiate parameters in ω_t, repetitive parameter values are selected by the corresponding repetitive ratios from $\Psi_{t,t-k,m}$ (of ω_{t-k}) to $\Psi_{t,t-1,m}$ (of ω_{t-1}) (*line 4-9*). Specifically, for h_m in ω_t, the number of repetitive parameter values $\mathcal{C}_{i,m}$ $w.r.t$ the ith previous referenced window ω_{t-i} is calculated based on the repetition ratio $\Psi_{t,t-i,m}$ and the total cardinality of the parameter space of h_m, i.e., $|\Phi_{t,m}|$. The candidate parameter values selected from ω_{t-i} should be the new values not in $\Omega_{t,m}$ (*line 7-8*). After selecting the parameter values for repetition from the

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

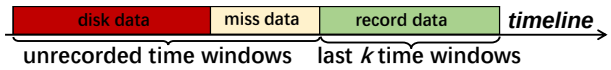


Fig. 7. An example of Data Access Distribution in timeline

previous windows, the other parameter values are generated to fill h_m of the current window ω_t to meet its total cardinality requirement. These new parameter values are all generated by column generator on *D-Dist* (*line 12*) and not in $\Omega_{t,m}$ (*line 13-15*). The generation complexity of repetition cardinalities for a time window in Algo. 2 is $O(k \cdot \mathcal{C})$ with k and $\mathcal{C}$ as the backward length and the average cardinality for parameters in each time window.

The backward length k controls the number of windows where the repetitive parameter values come. A small k increases the deviation of *cache hit ratio*, while a large one leads to a high cost in storage and computation. To trade off storage cost against deviation, we should provide a way to find a proper k for an acceptable deviation with a relatively low cost. Suppose that the database size is d, the cache size is n, and the size of data recorded by our algorithm is $k \cdot \mathcal{C}$. Since data with a higher access probability will be more likely to be accessed recently, DBMSs keep data accessed recently in the buffer [23]. We split all data into three parts, as shown in Figure 7. The first part of data is accessed most recently and recorded in our algorithm, named *record data*. *Record data* is stored in the buffer and has the highest access probability. The second part of data, whose size is $(n - k \cdot \mathcal{C})$, is also in the buffer but is not recorded in Algorithm 2, named *miss data*. The last part is the data in the disk, named *disk data*. The access probability of each data in three parts decreases sequentially. Since we only record *record data*, we cannot tell *miss data* and *disk data* during simulation. Thus, *miss data* generated may have a different access probability than that in the production environment, resulting in different *cache hit ratios*. The access probability of *miss data* is noted as P_{prod} in production; it is noted as P_{gen} in the simulation. And the deviation of *cache hit ratios* is noted as δ. Theorem 1 models the expectation deviation $\bar{\delta}$ of *cache hit ratio*.

THEOREM 1. *Suppose that the probability sum of top x data with the highest access probability is $cdf_T(x)$, and the expacted probability sum of any y data is $E_{\sum_y}$. When we instantiate a parameter p to access a database based on our algorithm, the expectation deviation of* cache hit ratio $\bar{\delta} = (1 - cdf_T(k \cdot \mathcal{C}))(cdf_T(n) - cdf_T(k \cdot \mathcal{C}) - E_{\sum n-k \cdot \mathcal{C}})$.

Proof. In Algorithm 2, there are two instantiation methods for p. The first one is to instantiate p to access *record data* (line 7). Since we generate *record data* as same as in production, there is no deviation, whose probability is $P(p \in \{\text{record data}\}) = cdf_T(k \cdot \mathcal{C})$. The second one is instantiating p based on the data access distribution (line 11), so accessing data in the buffer has the same probability as the access probability of *miss data*.
$\therefore$ The deviation of cache hit ratio between the production environment and the simulation is $P_{prod} - P_{gen}$.
$\therefore \delta = (1 - P(p \in \{\text{record data}\}))(P_{prod} - P_{gen})$
$\because$ In production, *miss data* has the highest access probability except *record data*.
$\therefore P_{prod} = cdf_T(n) - cdf_T(k \cdot \mathcal{C})$.
$\because$ Algorithm 2 generates *miss data* based on the access distribution.
$\therefore$ The average P_{gen} is $\bar{P_{gen}} = E_{\sum(n-k \cdot \mathcal{C})}$.
$\therefore \bar{\delta} = (1 - cdf_T(k \cdot \mathcal{C}))(cdf_T(n) - cdf_T(k \cdot \mathcal{C}) - E_{\sum n-k \cdot \mathcal{C}})$ □

We provide the calculation of $E_{\sum_y}$ in Appendix A. According to Theorem 1, $\bar{\delta}$ is related to recorded data size $k \cdot \mathcal{C}$ and the difference between $k \cdot \mathcal{C}$ and the cache size n. When we count more data, i.e., increasing $k \cdot \mathcal{C}$, $\bar{\delta}$ is reduced. In contrast, a large gap between $k \cdot \mathcal{C}$ and n makes an inaccurate *cache hit ratio*. Since $cdf_T(x)$ and $E_{\sum_y}$ depend on the data distribution, $\bar{\delta}$ can be calculated for any given distribution. With a similar analysis, Theorem 2 shows the upper bound of deviation for the common *Zipfian* data access distribution in applications [24].

THEOREM 2. *Suppose that the parameter p accessing the database follows Zipfian distribution, i.e., $p \sim Zipf(\alpha)$. The upper bound of δ is:*

$$\delta \leqslant (\sum_{i=1}^{d} i^{-\alpha})^{-1} \cdot (\sum_{i=k \cdot \mathcal{C}}^{d} i^{-\alpha}) \cdot (\sum_{i=k \cdot \mathcal{C}}^{n} i^{-\alpha} - \sum_{i=d-n+k \cdot \mathcal{C}}^{d} i^{-\alpha})$$

Proof. $\because p \sim Zipf(\alpha) \rightarrow cdf_T(x) = \frac{\sum_{i=1}^{x} i^{-\alpha}}{\sum_{i=1}^{d} i^{-\alpha}}$.
$\because$ The worst situation (the largest deviation) happens when the data generated has the lowest access probability, i.e., $P_{gen} = \sum_{i=d-(n-k \cdot \mathcal{C})}^{d} i^{-\alpha}$
$\therefore, \delta \leqslant (\sum_{i=1}^{d} i^{-\alpha})^{-1} \cdot (\sum_{i=k \cdot \mathcal{C}}^{d} i^{-\alpha}) \cdot (\sum_{i=k \cdot \mathcal{C}}^{n} i^{-\alpha} - \sum_{i=d-n+k \cdot \mathcal{C}}^{d} i^{-\alpha})$ □

In summary, Theorem 1 provides the expectation deviation of *cache hit ratio*. Theorem 2 provides the upper bound of δ for a given k with *Zipfian* data access distribution. These theoretical analyses construct the relationship between k and δ, which enhances the usability of $Lauca$. Based on it, we plot the relation between deviation δ and the backward length k by varying database size, cache size, and skew factor in Appendix A. We find that: 1) Increasing k can effectively reduce deviation δ. When k=6, δ is less than 10% in average. 2) For a high skew workload or a large database, i.e., a large α or d, a relatively small k can help to reach a low deviation. 3) For a large n (in-memory database), it has no further requirement of k. These results can guide the selection of k.

V. Partition Access Distrbution

In distributed DBMSs, partitioning and sharding strategies are vital for enhancing performance by parallel processing. However, the mapping strategy between logical partitions and physical nodes is invisible to users, which challenges us to simulate the workload characteristics from the view of physical nodes. As shown in Figure 2, the major workload characteristics influencing performance are on *Transaction Layer* and *Storage Layer*. On *Transaction Layer*, scheduling is launched based on logical partitions instead of physical nodes [25]; on *Storage Layer*, accessing partitions among physical nodes leads to high network I/O. So, inaccurate

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

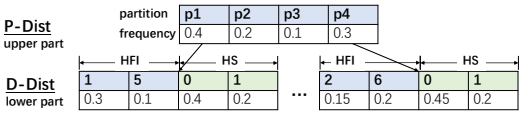


Fig. 8. The structure for *P-Dist*

physical access distribution simulation can cause performance deviations. To address this challenge, we propose **P**artition access **D**istribution (i.e., *P-Dist*) to ensure the distributed access characteristics. It quantifies partition access distribution by the *parameter-level* inner partition access distribution and the *transaction-level* inter partition access distribution.

A. Partition Access Distribution

If a table has a partition key, the partition rule divides data into different logical partitions by applying the partition rule on the column where the partition key is located. Specifically, *Data Characteristics Extractor* (in Figure 3) is used to extract the schema statistics into *H*, including the partition information (i.e., partition key and partition rule). Note that the partition rule is usually applied to the key column and is consistent with the access parameter as in TPC-C. Based on the partition rule, e.g., Hash, we obtain the partition distribution statistics for parameters/transactions, which are used to guide $Lauca$ to instantiate transaction templates.

To describe *P-Dist*, we design a two-level hierarchical access distribution organization structure, as shown in Figure 8. The upper part is *P-Dist*, where each entry represents a partition P_i together with its access probability. For example, 40% access is located in partition P_1. The low part is *D-Dist* representing the access distribution to data in each partition, defined by HFI and HS as in § IV-B.

B. Partition-aware Workload Generation

Parameter-level P-Dist specifies how a single operator accesses partitions. In a transaction, the number of its accessed partitions specifies **P**artition **C**ardinality **C**onstraint (*PCC*), which imposes *Transaction-level partition access cardinality* requirement. Take TX_1 in Figure 1 for example. If t_2 is the partition key and its related parameters, i.e., $p_{3,2}, p_{4,1}$ and $p_{5,2}$, locate in two partitions with probability 80%, we then impose a *PCC* requirement to guarantee $p_{3,2}$, $p_{4,1}$ and $p_{5,2}$ selected from two partitions with probability 80%. *PCC* can be easily constructed by counting the accessed partitions of the parameters. Specifically, *Data Characteristics Extractor* iterates over all parameters in a transaction and obtains the *partition access cardinality* based on the partition rule. For a transaction that has n parameters, the complexity is $O(n)$.

C. Physical Sharding in Distributed Databases

Different DBMSs may have different physical sharding implementations, which are invisible to users. It is impossible to directly depict how the workload accesses physical nodes. The mapping between logical partitions and physical nodes is determined by the implementation of a DBMS instead of the applications. Since the access distribution on logical partitions is guaranteed by *P-Dist*, a consistent physical data access distribution is maintained indirectly by the same mapping relationship of the DBMS. Usually, there are two partition sharding strategies. Some distributed databases, e.g. OceanBase [26], specify a static mapping between logical partitions and physical nodes, which does not change during workload execution; some other distributed databases, e.g., TiDB [27], dynamically rebalance shardings by scheduling logical partitions based on the *Data Access Distribution* of workload to eliminate hotspots. For the database with a predefined static partition rule, we adopt the same rule to control parameter partition access; for the dynamical rebalance mechanism, warming up the database with the generated workload allows the database to schedule the logical partitions identically, ensuring the same physical access distribution.

VI. Workload Generation

Given *Transaction Logic* of each transaction template and *Access Distribution* of each parameter, *Workload Generator* starts generating synthetic workloads. We present the details of workload generation from three levels: thread model, transaction execution, and parameter instantiation.

Thread Model. Users can configure multiple test nodes for deploying workload generators. On each node, we start a *JVM* process with the specified number of test threads (i.e., test clients), and establish a separate database connection for each test thread to concurrently launch workloads. $Lauca$ supports generating extended synthetic workloads according to the user's requirement specified by Scale Factor (SF), based on the *throughput info* from workload statistics. It is achieved by controlling *think time* among transaction requests. When SF=1, we simulate the workloads with the same *throughput* as in the production environment. When SF>1, we decrease *think time* to speed up the generation and then we can scale out the *throughput* of real application for the target one, i.e., *throughput* $\times$ SF. When the required *throughput* exceeds the maximum *throughput* of current test threads, i.e., *think time=0*, we shall increase the client nodes or generation threads.

Transaction Execution. The test thread invokes different kinds of transactions according to their proportions extracted from workload traces and SF specified by users. The statistics can be adjusted in each time window. For the execution of a specific SQL operation, we first instantiate all the symbolic parameters and then send the operation with concrete parameter values to the test database. After operation execution, the result set and the parameters are maintained temporarily for the generation of other parameters in subsequent operations within the same transaction instance.

Parameter Instantiation. When instantiating the parameters in transaction templates, we should first guarantee the consistency of *Transaction Logic*, and then ensure *Access Distribution* of synthetic workloads for both data and partition. For a parameter, *Case 1:* if it is only involved with a *BR* dependency, the value of this parameter can be calculated directly based on the increment Δ and the value of the dependent one. *Case 2:* If a parameter is involved with a dependency of *ER, IR, LR* outside the loop structure, we instantiate the parameter according to the existence probabilities. *Case 3:* If a parameter is involved in a loop structure, for the non-first-round of execution, we instantiate the parameter based on the

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

detected dependency in a probability. But for the first round of execution or no such detected dependency, we instantiate the parameter as in *Case 1* and *Case 2*. For all other situations, *Data Access Distribution* and *Partition Access Distribution*, if any, are used to instantiate the parameter.

Overall, transaction execution and parameter instantiation are independent of each other for all test threads, so our workload generator can be deployed on multiple nodes to generate the high *concurrency* or *throughput* synthetic workloads with the desired workload characteristics and configurations.

We have a thorough discussion about $Lauca$ in Appendix B: the proposed technology is applicable for databases with different concurrency control protocols and isolation levels; it can protect sensitive data to a great degree.

VII. Experiments

Environment. We evaluate $Lauca$ on one centralized database, i.e., MySQL (v5.7.24), and two distributed databases with different partition organization mechanisms, i.e., TiDB (v6.5.0) and OceanBase(v4.0.0). When we launch experiments on MySQL, we deploy two separate servers one for the client and the other one for MySQL. Each server has 2 Intel Xeon Silver 4110 @ 2.1 GHz CPUs, 120GB memory, 4TB HDD disk configured by RAID-5, and 4GB RAID cache. We deploy TiDB on a 5-node cluster with 3 TiKV servers, 1 PD server, and 1 monitor server. Each TiKV server has 8 Intel Cascadelake 6248R @ 3.0GHz CPUs and 16GB of RAM. All servers are connected using Gigabit Ethernet. OceanBase is deployed on the same 5-node cluster.

Workloads. TPC-C [6], SmallBank [7] and YCSB [8] play the role of three different 'real' application scenarios[1] by running the workload on a given database. TPC-C is one of the most widely used industrial-level OLTP benchmarks for a complex order processing application. SmallBank abstracts the operations from banking applications. Since YCSB has less *Transaction Logic*, it is used to demonstrate the performance influence of *Access Distribution* without the complex influence of transaction processing. We use OLTP-Bench [9] to generate workloads in TPC-C, SmallBank and YCSB.

Settings and Setup. In *Transaction Logic* extraction, we set the number of transaction instances K and the group number of transaction instances N_l as K=N_l=10^4. For *Access Distribution* extraction, the number of items N_H in HFI and the number of intervals I in HS are all set to 50, and the time window size is set to 1 second. According to the experimental simulation in Appendix A, we set the backward length k as 6, where the *cache hit ratio* is less than 10% in average. Workload generators, i.e., $Lauca$ and OLTP-Bench, are deployed on a single server without DBMS. To simplify the demonstration of our simulation technology and avoid the influence of other factors, we run distributed databases only when showcasing the simulation result affected by or related to distributed characteristics (§VII-A,§VII-D and §VII-E); for other cases, we run MySQL for experiments. For space limitation, we leave some experiments about simulation fidelity, workload adaptability, and data protection in Appendix C.

[1]Result reproduction can be done according to the steps in README, which is an example adaption of $Lauca$ for TiDB at https://github.com/luyiqu/Lauca.

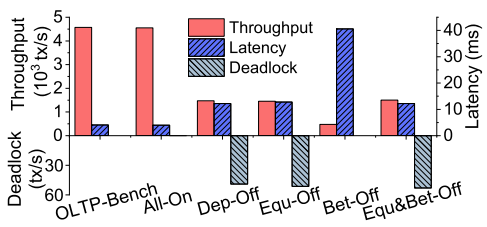


Fig. 9. The impact on transaction throughput, deadlock and latency

The simulation *fidelity* is measured by performance deviations, i.e., $\frac{|sp-rp|}{rp}$, when running the synthetic workloads and real workloads on the same database with the simulated performance(sp), and the real performance(rp). We demonstrate performance on *TPS*, *latency*, and *resource usage*.

The data size is represented by scale factor (SF). TPC-C and SmallBank require the concurrent thread number (TH) is equal to SF. By default, we set SF/TH=20. TPC-C and SmallBank have data sizes about 1 GB and 10 MB, respectively (dependent on the implementation of DBMSs). YCSB does not require SF equals TH, we generate the same data size of YCSB as TPC-C, i.e., about 1 GB (100M rows).

A. Fidelity of Synthetic Workloads

Table II shows performance deviations between synthetic workloads generated by $Lauca$ and real workloads generated by OLTP-Bench under different SFs. We execute all workloads on MySQL, TiDB, and OceanBase. First, we present the simulation result on a centralized database, i.e., MySQL. On MySQL, our two synthetic workloads perform well for both *throughput* and *latency*, whose deviations are lower than 5.8% and 3.8% respectively. For resource consumption, i.e., *CPU* and *Disk* usages by the workloads, the synthetic workloads and the real workloads perform almost the same on MySQL, and the maximum deviations are 5.9% and 3.0% for *CPU* and *Disk I/O* respectively.

On distributed databases, i.e., TiDB and OceanBase, when varying SFs, the deviations of running performance from TPC-C and Smallbank are less than 3% for *throughput*, 2% for *latency*; for resource consumption, the deviations are less than 3% for *CPU*, 5% for both *Disk* and *Network I/O*. The results verify the high fidelity of synthetic workloads compared to the real workloads. More details are in Appendix C-1.

B. Exploring Transaction Logic

In this section, we demonstrate the impact of *Transaction Logic* on *transaction semantics*, *transaction conflict intensity*, and *deadlock possibility* of synthetic workloads on MySQL. Though *Transaction Logic* consists of *transaction structure information* and *parameter dependency information*, in which structure information, such as the number of loops, has an obvious impact on performance, here we mainly explore the influence of parameter dependency on database performance.

We run the real workload and the synthetic one of TPC-C generated by OLTP-Bench and $Lauca$ to explore the impact of *Transaction Logic*, respectively. Notice that when we turn

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

TABLE II
PERFORMANCE DEVIATIONS (%) BETWEEN SYNTHETIC WORKLOADS AND REAL WORKLOADS UNDER DIFFERENT DATABASES

Deviation(%)		Throughput				Latency				CPU				Disk I/O				Network			
Scale Factor (SF)		10	20	50	100	10	20	50	100	10	20	50	100	10	20	50	100	10	20	50	100
MySQL	TPC-C	2.3	2.2	1.2	1.7	3.0	2.3	0.6	1.6	1.8	1.6	**5.9**	0.6	**3.0**	2.0	0.9	0.1	/	/	/	/
	SmallBank	3.2	1.4	1.8	**5.8**	0.4	0.3	**3.8**	0.0	0.3	1.4	0.5	0.2	1.0	0.9	1.5	0.1	/	/	/	/
TiDB	TPC-C	0.9	2.4	1.9	**2.7**	0.6	1.2	**2.0**	0.4	**2.5**	0.7	3.2	0.2	2.3	1.7	1.3	0.2	1.4	3.1	1.9	0.4
	SmallBank	0.1	0.5	0.2	0.2	0.0	1.4	0.6	0.6	0.9	0.4	1.7	1.1	3.4	1.6	1.8	**4.3**	3.6	2.2	2.6	**3.9**
OceanBase	TPC-C	0.8	2.3	0.1	0.5	0.5	0.5	1.6	1.5	0.8	**2.1**	0.0	2.0	3.3	2.9	**4.4**	3.0	0.4	0.2	2.1	1.7
	SmallBank	0.2	**2.4**	2.2	1.9	1.2	1.0	0.1	**2.0**	0.3	2.1	0.1	1.1	1.9	2.4	3.5	3.7	1.3	0.3	1.1	**4.7**

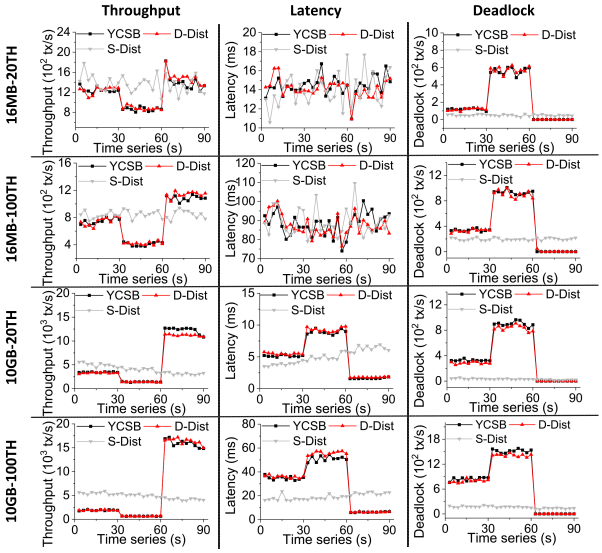
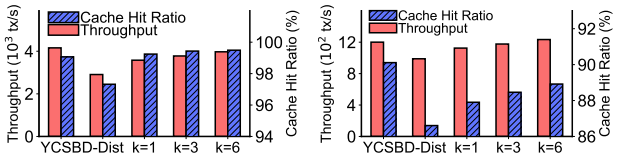


Fig. 10. Exploring S-Dist and D-Dist for skewed and dynamic workloads



(a) Buffer Size=10GB (b) Buffer Size=16MB

Fig. 11. Exploring C-Dist for continuous workloads

off the dependency, the instantiation of parameters breaks the transaction semantics, causing some transaction execution failure (details see Appendix C-2). Here we focus on three types of transactions successfully executed, i.e., *Payment*, *OrderStatus*, and *StockLevel* transactions. In Figure 9, we present transaction *throughput*, *latency* and *deadlock throughput*. We have five groups of experimental results for $Lauca$ by turning off different switches of *Transaction Logic*. The results show that when we turn off *ER* dependency (*Equ-Off*), the *throughput* decreases significantly, the *latency* increases markedly, and a lot of deadlocks occur. This is because three pairs of read and write operations involve the same record in Payment. Without *ER*, reads and writes are likely to operate on different records and increase the likelihood of transaction conflicts and deadlocks. When we turn off *BR* dependency (*Bet-Off*), the *latency* increases drastically, and the *throughput* is very low. This is because the scan operations in *StockLevel* involve a large amount of unexpected data instead of about 20 records in the real workloads. When we take both *Equ-Off* and *Bet-Off*, the scan operation does not read a large amount of data due to the role of *C-Dist*, so the performance metrics of *Dep-Off*, *Equ&Bet-Off* and *Equ-Off* are similar. Overall, *Transaction Logic* enables synthetic workloads to have the same *transaction conflict intensity*, *deadlock possibility*, and *scan data volume* as real workloads have.

C. Exploring Data Access Distribution

We demonstrate the simulation ability of $Lauca$ to the *skewness*, *dynamics* and *continuity* of data access, namely *S-Dist, D-Dist* and *C-Dist*. Experiments are carried out on MySQL with a test table from YCSB sized 10^8 (100M rows). We have two concurrencies of database requests, i.e., TH=20 and TH=100. *Innodb_buffer_pool_size* of MySQL has two settings, i.e., a small size of 16MB and a large size of 10GB, while the first one simulates limited database buffer and the second one simulates a large buffer.

The evaluation workload in Figure 10 consists of five pairs of read-write operations, each of which reads a record first and then updates it. The extended YCSB workload runs for 90 seconds, which is divided into three phases, and the data requests in each phase are randomly selected upon 10^5 records. In the first phase, *Data Access Distribution* is a *Zipfian* distribution with parameter $\alpha = 1$; the second phase is still a *Zipfian* distribution with parameter $\alpha = 1.2$; and the third phase is a *Uniform* distribution.

We have four different settings for *S-Dist* and *D-Dist* $w.r.t$ buffer pool size and concurrency in Figure 10, i.e., $16MB$-$20TH$, $16MB$-$100TH$, $10GB$-$20TH$ and $10GB$-$100TH$. The results show the global *S-Dist* is not working well (grey lines in Figure 10), which is defined for the whole running period and does not take the dynamic changes of workloads into account. *D-Dist* is represented by a list of *S-Dist* for each time window, which has a consistent performance with the real workloads on dynamicity evaluated by *throughput*, *latency* and *deadlock* in both small and large *Innodb_buffer_pool_size* and concurrencies. Specifically, in a small *Innodb_buffer_pool_size*, the performance of both the real and the synthetic workloads fluctuates drastically, due to a low *cache hit ratio* and a frequent *page switching*; in a large one, both of them have relatively stable performance. In addition, *throughput* becomes smaller with a large concurrency due to higher contention in a *Zipfian* access distribution.

In Figure 11, we further explore the performance for *C-Dist*

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

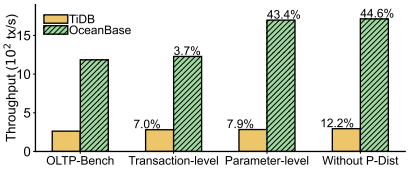


Fig. 12. Throughputs by different partition simulation methods

by looking back at different numbers of windows (k) under different buffer pool sizes. We compare *throughput* and *cache hit ratio* for the YCSB workload. The simulation results for $Lauca$ with *D-Dist* and *C-Dist* (labeled by backward length k) are presented. Specifically, *cache hit ratio* is calculated by:

$$cache_hit_rate{=}1 - \frac{Innodb_buffer_pool_reads}{Innodb_buffer_pool_read_requests}$$

Innodb_buffer_pool_reads in MySQL [28] is the number of logical reads that InnoDB cannot get from the buffer pool, i.e., read from disk. *Innodb_buffer_pool_read_requests* is the number of all logical reads. From the results, we can see that *D-Dist* has a 25% lower *cache hit ratio* and *throughput*. This is because *D-Dist* is not good at catching the continuity of data access distribution, causing a more frequent *page switching*. For *C-Dist*, increasing k, i.e., longer backward length, $Lauca$ approaches YCSB in both *throughput* and *cache hit ratio*. We can see that the simulation effectiveness is sensitive to k. Meanwhile, a large buffer size can itself reduce the deviation, as mentioned in §IV-C. Therefore, the deviation in Figure 11a is less than that in Figure 11b with the same k. However, increasing k is effective for catching *C-Dist* and achieves a better performance simulation.

D. Distributed Characteristics of Workloads

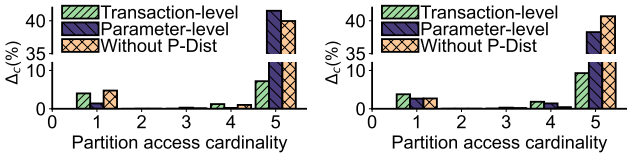
Partition Access Distribution (*P-Dist*) describes and controls the *distributed transaction ratio*, which has a significant impact on distributed database performance [15], [17], [25]. We use *partition access cardinality* c (> 0) to represent partitions accessed by the current transaction. When c=1, the transaction is located in a single partition. Given a c, *partition cardinality deviation* Δ_c between the simulated workload sp_c and real workload rp_c is calculated by

$$\Delta_c = \frac{|rp_c - sp_c|}{rp_c}$$

We use the sum of Δ_c to measure the fidelity of the simulation $w.r.t$ different *partition access cardinalities*.

To evaluate the simulation effect, we run TPC-C workloads in 100 seconds on both TiDB and OceanBase, where data is hash-partitioned by Warehouse ID on five logical partitions (corresponding to 5 physical nodes). We compose a transactional workload from TPC-C to better investigate the effect of *P-Dist*. In TPC-C, only *NewOrder* and *Payment* access data from multiple partitions, while *Payment* accesses two partitions. We use the workload with *All NewOrder* transactions, which has different partition access cardinalities and more obvious deviations compared to *Payment*. More evaluations using various workloads are presented in Appendix C-4.

We study two kinds of *Partition Access Distribution*, i.e., *parameter-level* and *transaction-level*. *Parameter-level* (resp. *transaction-level*) simulates *P-Dist* for parameters (resp. transactions). We accumulate deviations of *throughputs* from the two levels of *P-Dist* as well as the case without *P-Dist* in Figure 12. We find that *transaction-level* partition access control has much better fidelity of *throughputs* on TiDB and OceanBase with deviations 7.0% and 3.7% respectively. It is because *Transaction-level* considers the interactions among operations in the same transaction, while *Parameter-level* and *Without P-Dist* take each parameter individually.



(a) TiDB (b) OceanBase

Fig. 13. Deviations of P-Dist for different partition access cardinalities

Next, to expose the details of the interactions of operations, we plot *partition cardinality deviation* for each candidate cardinality in Figure 13. We find that *Transaction-level* dominates the other two ways, i.e., *Parameter-level* and *Without P-Dist*, in deviations under a higher *partition access cardinality* requirement, i.e., c=5. Thus, individually treating each parameter causes a lower *partition access cardinality* in a transaction than expected, resulting in a higher *throughput* in Figure 12. In summary, *transaction-level* partition access simulation has a positive impact on simulation fidelity.

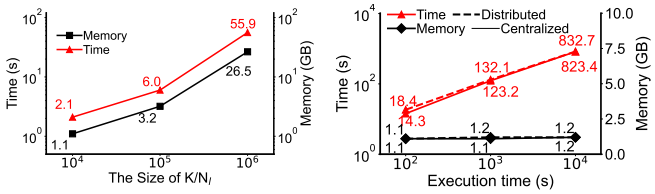
E. Efficiency of Lauca

Here we use TPC-C workload traces generated by OLTP-Bench to study the performance of $Lauca$. Figure 14a shows the execution time and memory consumptions of *Transaction Logic* extraction under different sizes of K(=N_l). The performance of *Transaction Logic* extraction remains unaffected by DBMS types, so we use the traces from MySQL here. Memory is used for logging transaction instances and increases linearly with K. Though a larger K can include more transaction instances and result in more accurate *Transaction Logic*, the business logic in an application keeps stable; therefore, a smaller K is usually good enough. The experiments above are all conducted with K and N_l set to 10^4 and have proved the high fidelity of generated workloads. When K=10^4(=N_l), the extraction time is only 2.1 seconds and the memory consumption is 1.1 GB.

Figure 14b presents the time and memory consumptions of extracting *Access Distribution* on both a centralized database (MySQL) and a distributed database (TiDB). In MySQL, *C-Dist* is extracted; in TiDB, *P-Dist* is covered to control distributed accessing as well. The results show that the extraction time is linear with the size of workload traces with constant memory consumption. It is because *C-Dist* is a window-based *Partition Access Distribution* extraction, and at least traces of k continuous time windows should be kept in memory and moved out after processing. In Figure 14b, when running TPC-C in MySQL with the *throughput* as 3.61×10^3 TPS, the workload traces of 10^4 seconds (=2.8 hours) is 33.8GB, which is extracted within 10 minutes and 1.2GB. Workload traces can

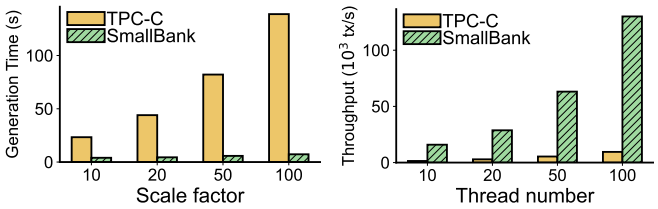
Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116



(a) Transaction Logic (b) Access Distribution

Fig. 14. Characteristics extraction



(a) Database Generation (b) Workload Generation

Fig. 15. Generation time and efficiency

be divided into time-ordered groups for distribution analysis to avoid out-of-memory problems. For traces from TiDB, it first maps parameters to logical partitions in a lightweight way; then it calculates *C-Dist* for each logical partition, which dominates the Access distribution extraction time.

We demonstrate the *scalability* of *Lauca* as well. The time cost of database generation grows linearly with the increase of data size (SF), as in Figure 15a. When SF=100, *Lauca* generates TPC-C database within 2 minutes with well-designed parallel data generation. Workload is generated while loading in parallel. Figure 15b shows that the throughput of workload generation linearly scales up with more threads. In this way, *Lauca* meets the requirement of generating high concurrency or throughput workloads.

F. Summary to Experiment

In summary, the experiments have shown the effectiveness and efficiency of *Lauca* in simulating an application scenario based on the transaction templates and statistics of both data and transactions. Firstly, we have verified that *Lauca* has a low simulation deviation ($< 6\%$) for all metrics, i.e., good fidelity for an application simulation. Secondly, *Lauca* can generate a large scale of data/workload to satisfy scalability and extensibility. Thirdly, the design of *Lauca* is independent of databases, which can be applied to applications on either centralized or distributed DBMSs.

VIII. Related Work

Classic benchmarks are proposed for database performance evaluation. There are TPC-H, TPC-DS, and SSB [29] for OLAP applications and TPC-C, TPC-E, and SmallBank [7] for OLTP applications. In addition, CH-benCHmark [30], HTAPBench [31] and OLxPBench [32] can provide a unified assessment for hybrid transaction/analytical processing (HTAP) systems. However, the workloads of these standard benchmarks are abstractions for a class of applications and too general to evaluate DBMS performance for a specific one.

Simulation of applications is a key necessity to determine the efficiency and usefulness of a specific database for a specific task. Not having a way to simulate a production system in an open environment may bottleneck the development of both applications and databases. DIAMetrics [5] designs an application benchmark framework for query engines instead of transaction processing engines. There are workload-aware data and query generators [2], [3], [33]–[35] for database performance evaluation of OLAP applications. Workload analyzers [36] are designed to study and better understand the application workloads, but neither can generate synthetic workloads. Jeong [37] proposes a workload generator for simulating a realistic hardware resource consumption status. NoWog [38] introduces a workload description language for generating synthetic workloads benchmarking NoSQL databases. The latest work innovatively introduces learning approaches to workload simulation. It learns the time dependency of real workload to simulate cloud workload for evaluating resource management policy [39]; Stitcher [40] synthesizes a workload based on the existing benchmarks, which yields the same performance profile. These approaches have different design targets from *Lauca* and they can not simulate an application scenario from the transaction templates and performance-related statistics.

IX. Conclusion and Future Work

Lauca is a transactional workload generator for application-driven database performance evaluation. It proposes to simulate OLTP applications on workload similarity and guarantee the latent execution semantics of transactions represented by ***Transaction Logic***, ***Data Access Distribution***, and ***Partition Access Distribution***. Then it simplifies the data generation in randomized ways satisfying the fundamental data characteristics. It successfully avoids the trouble in traditional methods based on data simulation, which has great challenges in data protection and performance similarity. After simulating various applications on different databases, our experiments show that *Lauca* generates high-quality synthetic workloads.

Though we have no exhaustive definitions of parameter dependencies in *Transaction Logic*, it has been found that *Transaction Logic* together with *Data Access Distribution* and *Partition Access Distribution* can well catch the runtime workload characteristics on both centralized and distributed databases. By increasing the window size of *C-Dist*, we can portray more accurate data access dependency among transactions. Notice that when introducing new dependencies or access distributions, *Transaction Logic* and *Access Distribution* can be easily extended in generating synthetic workloads.

Acknowledgments

Rong Zhang, CaiPeng, Weining Qian and Aoying Zhou are supported by National Science Foundation of China (No. 62072179), the joint project with Key Laboratory of Performance and Reliability Evaluation of Basic Hardware and Software, Ministry of Industry and Information Technology, and joint project with Huawei Technologies Co., Ltd. Thanks to the great effort made by Yuming Li, Ting Chen and Ke Shu, who have contributed to the first version of *Lauca*.

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Knowledge and Data Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TKDE.2024.3360116

REFERENCES

[1] M. Seltzer *et al.*, "The case for application-specific benchmarking," in *HotOS*, 1999, pp. 102–107.
[2] Q. Wang *et al.*, "A Scalable Query-Aware Enormous Database Generator for Database Evaluation," in *TKDE*, vol. 35, no. 5, 2023, pp. 4395–4410.
[3] E. Lo *et al.*, "Mybenchmark: generating databases for query workloads," in *VLDBJ*, vol. 23, no. 6, 2014, pp. 895–913.
[4] A. Sanghi, "HYDRA: A Dynamic Approach to Database Regeneration," Ph.D. dissertation, Indian Institute of Science Bangalore, 2022.
[5] S. Deep *et al.*, "DIAMetrics: benchmarking query engines at scale," in *VLDB*, vol. 13, no. 12, 2020, pp. 3285–3298.
[6] "TPC-C benchmark," http://www.tpc.org/tpcc/.
[7] M. Alomari *et al.*, "The cost of serializability on platforms that use snapshot isolation," in *SIGMOD*, 2008, pp. 576–585.
[8] B. F. Cooper *et al.*, "Benchmarking cloud serving systems with YCSB," in *SoCC*, 2010, pp. 143–154.
[9] D. E. Difallah *et al.*, "OLTP-Bench: An extensible testbed for benchmarking relational databases," in *VLDB*, vol. 7, no. 4, 2013, pp. 277–288.
[10] J. Yang *et al.*, "SAM: Database Generation from Query Workloads with Supervised Autoregressive Models," in *SIGMOD*, 2022, p. 1542–1555.
[11] A. Pavlo, "What are we doing with our lives?: nobody cares about our concurrency control research," in *SIGMOD*, 2017, pp. 3–3.
[12] L. Qu *et al.*, "Application-Oriented Workload Generation for Transactional Database Performance Evaluation," in *ICDE*, 2022, pp. 420–432.
[13] H. Lan *et al.*, "A survey on advancing the dbms query optimizer: Cardinality estimation, cost model, and plan enumeration," in *DSE*, vol. 6, no. 1, 2021, pp. 86–101.
[14] N. Krivokapić *et al.*, "Deadlock detection in distributed database systems: a new algorithm and a comparative performance analysis," in *VLDBJ*, vol. 8, no. 2, 1999, pp. 79–100.
[15] R. Harding *et al.*, "An evaluation of distributed concurrency control," in *VLDB*, vol. 10, no. 5, 2017, pp. 553–564.
[16] L. Qu *et al.*, "Are current benchmarks adequate to evaluate distributed transactional databases?" in *BenchCouncil*, vol. 2, no. 1, 2022, p. 100031.
[17] C. Curino *et al.*, "Schism: a workload-driven approach to database replication and partitioning," in *VLDB*, vol. 3, no. 1-2, 2010, pp. 48–57.
[18] H. Zhang, L. Qu, Q. Wang, R. Zhang, P. Cai, Q. Xu, Z. Yang, and C. Yang, "Dike: A Benchmark Suite for Distributed Transactional Databases," in *SIGMOD*, 2023, pp. 95–98.
[19] S. Harizopoulos *et al.*, "OLTP through the looking glass, and what we found there," in *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*, 2018, pp. 409–439.
[20] H. Garcia-Molina *et al.*, *Database system implementation*. Prentice Hall Upper Saddle River, NJ:, 2000, vol. 672.
[21] Y. Li *et al.*, "Touchstone: generating enormous query-aware test databases," in *USENIX ATC*, 2018, pp. 575–586.
[22] A. J. Bonner *et al.*, "A logic for programming database transactions," in *Logics for databases and information systems*, 1998, pp. 117–166.
[23] E. J. O'Neil *et al.*, "The LRU-K page replacement algorithm for database disk buffering," in *SIGMOD*, vol. 22, no. 2, Jun. 1993, pp. 297–306.
[24] Y. Yang *et al.*, "Write Skew and Zipf Distribution: Evidence and Implications," in *TOS*, vol. 12, no. 4, Aug. 2016, pp. 1–19.
[25] Q. Lin *et al.*, "Towards a non-2pc transaction management in distributed database systems," in *SIGMOD*, 2016, pp. 1659–1674.
[26] Z. Yang *et al.*, "OceanBase: a 707 million tpmC distributed relational database system," in *VLDB*, vol. 15, no. 12, 2022, pp. 3385–3397.
[27] D. Huang *et al.*, "TiDB: a Raft-based HTAP database," in *VLDB*, vol. 13, no. 12, 2020, pp. 3072–3084.
[28] "MySQL :: MySQL 8.0 Reference Manual :: 5.1.10 Server Status Variables." [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/server-status-variables.html
[29] P. O'Neil *et al.*, "The star schema benchmark and augmented fact table indexing," in *TPCTC*, 2009, pp. 237–252.
[30] R. Cole *et al.*, "The mixed workload CH-benCHmark," in *DBTest*, 2011, p. 8.
[31] F. Coelho *et al.*, "HTAPBench: Hybrid transactional and analytical processing benchmark," in *ICPE*, 2017, pp. 293–304.
[32] G. Kang *et al.*, "OLxPBench: Real-time, Semantically Consistent, and Domain-specific are Essential in Benchmarking, Designing, and Implementing HTAP Systems," in *ICDE*, 2022, pp. 1822–1834.
[33] C. Binnig *et al.*, "QAGen: generating query-aware test databases," in *SIGMOD*, 2007, pp. 341–352.
[34] A. Arasu *et al.*, "Data generation using declarative constraints," in *SIGMOD*, 2011, pp. 685–696.
[35] A. Sanghi *et al.*, "Projection-compliant database generation," in *VLDB*, vol. 15, no. 5, 2022, pp. 998–1010.
[36] Q. T. Tran *et al.*, "Oracle workload intelligence," in *SIGMOD*, 2015, pp. 1669–1681.
[37] H. J. Jeong *et al.*, "A Workload Generator for Database System Benchmarks," in *iiWAS*, 2005, pp. 813–822.
[38] P. Ameri *et al.*, "NoWog: a workload generator for database performance benchmarking," in *DASC/PiCom/DataCom/CyberSciTech (2016)*, 2016, pp. 666–673.
[39] W. Lin *et al.*, "A GAN-Based Method for Time-Dependent Cloud Workload Generation," in *JPDC*, vol. 168, Oct. 2022, pp. 33–44.
[40] C. Wan *et al.*, "Stitcher: Learned Workload Synthesis from Historical Performance Footprints," in *EDBT*, 2023, pp. 417–423.



SiYang Weng is a PhD candidate student of East China Normal University (ECNU). He received a BS degree in Data Science and Engineering from ECNU in 2022. His current research interests include transaction processing, data management and database testing.



QingShuai Wang is a PhD candidate student of East China Normal University (ECNU). He received BS degree in Software Engineering from ECNU in 2019. His current research interests include transaction processing, data management and database benchmarking.



Luyi Qu is a master student of East China Normal University (ECNU). She received her BS degree of data science and big data technology from ECNU in 2020. Her research interests include database benchmark, application-oriented workload generation, and transaction processing.



Rong Zhang is a member of China Computer Federation. She received her PhD degree in computer science from Fudan University in 2007. She joined East China Normal University since 2011 and is currently a professor in the university. Her current research interests include distributed data management and database benchmarking.



Peng Cai is a member of China Computer Federation. He joined East China Normal University since 2015 and is currently a professor in the university. Previously, he worked at IBM China Research Lab and Baidu. His current research interests include high-performance transaction processing, and machine learning for database systems.



Weining Qian is a professor and dean of the School of Data Science and Engineering, East China Normal University. He is now serving as a standing committee member of Database Technology Committee of China Computer Federation. His research interests include transaction processing, database benchmarking, and massive data management.



Aoying Zhou is a professor on Computer Science at East China Normal University. He is now acting as a vice-director of ACM SIGMOD China and Database Technology Committee of China Computer Federation. His research interests include data management, transaction processing and database benchmarking.

Authorized licensed use limited to: Nanjing University. Downloaded on February 21,2024 at 09:22:05 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.